

PYTHON CHATBOT KNOWLEDGE BASE

Purpose: A structured Python knowledge source for a chatbot/RAG application.

RAG use: Load this PDF, extract text, split it into chunks, create embeddings, store vectors, retrieve relevant chunks, and provide those chunks as context to an LLM.

Coverage: Python basics, data types, strings, collections, control flow, functions, OOP, exceptions, files, modules, comprehensions, iterators, generators, decorators, regex, JSON, APIs, SQL, Flask, testing, debugging, common errors, coding patterns, and chatbot/RAG concepts.

1. Python Fundamentals

What is Python?

Python is a high-level, general-purpose programming language known for readable syntax and a large ecosystem.

Why is Python popular?

It has simple syntax, a large standard library, strong community support, cross-platform support, and libraries for web development, automation, data science, AI, and testing.

Is Python compiled or interpreted?

Python is commonly called interpreted. In CPython, source code is compiled to bytecode and executed by the Python virtual machine.

What is indentation?

Indentation defines code blocks in Python. Consistent indentation is required.

What is CPython?

CPython is the standard and most widely used implementation of Python.

2. Variables and Data Types

What is a variable?

A variable is a name referring to an object. Python does not require an explicit variable type declaration.

Common data types

int, float, complex, bool, str, list, tuple, set, dict, range, bytes, bytearray, and NoneType.

Dynamic typing

A variable can refer to objects of different types at different times.

Mutable objects

Lists, dictionaries, and sets are common mutable objects.

Immutable objects

Integers, floats, strings, tuples, and frozensets are common immutable objects.

type vs isinstance

type(x) returns the object's type. isinstance(x, T) checks whether x is an instance of T or a compatible subclass.

3. Input and Output

input()

input() reads a line and returns a string.

Convert input

Use `int(input())`, `float(input())`, or `map(int, input().split())` for numeric input.

print()

`print()` writes values to standard output.

f-string

f-strings insert expressions directly into a string, for example `f'Hello {name}'`.

4. Operators

Arithmetic

`+`, `-`, `*`, `/`, `//`, `%`, and `**` perform arithmetic. `/` is true division; `//` is floor division.

Comparison

`==`, `!=`, `>`, `<`, `>=`, `<=` compare values.

Logical

`and`, `or`, and `not` combine Boolean expressions.

Identity

`is` and `is not` check whether two references point to the same object.

Membership

`in` and `not in` test membership in a container.

5. Strings

What is a string?

A string is an immutable sequence of Unicode characters.

Indexing

Positive indexes start at 0. Negative indexes count from the end.

Slicing

`sequence[start:stop:step]` extracts part of a sequence; `stop` is excluded.

Useful methods

`upper()`, `lower()`, `title()`, `capitalize()`, `strip()`, `replace()`, `split()`, `join()`, `find()`, `startswith()`, `endswith()`, `count()`.

String mutation

Strings cannot be changed in place. Create a new string using slicing or string methods.

6. Lists

What is a list?

A list is an ordered, mutable collection that can contain duplicates and mixed types.

Methods

`append`, `extend`, `insert`, `remove`, `pop`, `clear`, `index`, `count`, `sort`, `reverse`, and `copy`.

append vs extend

`append` adds one object as one element. `extend` adds elements from an iterable.

List comprehension

A compact way to create lists: [expression for item in iterable if condition].

7. Tuples

What is a tuple?

A tuple is an ordered, immutable collection.

One-element tuple

Use (value,) with a comma. (value) is just a parenthesized expression.

Tuple unpacking

a, b = (10, 20) assigns tuple elements to variables.

Tuple vs list

Tuples are immutable; lists are mutable. Tuples can be hashable when all contained values are hashable.

8. Sets

What is a set?

A set is a collection of unique hashable elements.

Operations

add, update, remove, discard, pop, union, intersection, difference, and symmetric_difference.

remove vs discard

remove raises KeyError when absent; discard does not.

9. Dictionaries

What is a dictionary?

A dictionary stores key-value pairs. Keys must be hashable.

Methods

get, keys, values, items, update, pop, popitem, setdefault, and clear.

get vs []

d[key] raises KeyError for a missing key. d.get(key) can return None or a supplied default.

Dictionary comprehension

Example: {x: x*x for x in range(5)}.

10. Conditions and Loops

if/elif/else

Conditional blocks execute based on Boolean expressions.

Truthiness

0, None, False, empty strings, and empty containers are falsey.

for

A for loop iterates over an iterable.

while

A while loop repeats while its condition is true.

break

Stops the nearest loop.

continue

Skips the current iteration and continues with the next.

pass

Does nothing and acts as a placeholder.

range()

range(start, stop, step) generates an integer sequence-like object.

11. Functions

Function

A function is reusable code defined with def.

Parameters vs arguments

Parameters are names in a function definition; arguments are values supplied at the call.

return

return sends a value back and exits the function.

Default parameter

A parameter can have a default value, such as def greet(name='User').

***args**

Collects extra positional arguments into a tuple.

****kwargs**

Collects extra keyword arguments into a dictionary.

lambda

A small anonymous function containing one expression.

Scope

Python name lookup follows Local, Enclosing, Global, Built-in (LEGB).

12. Comprehensions

List

[x*x for x in numbers]

Set

{x*x for x in numbers}

Dictionary

{x: x*x for x in numbers}

Condition

[x for x in numbers if x % 2 == 0]

13. Object-Oriented Programming

Class

A class defines attributes and behavior for objects.

Object

An object is an instance of a class.

__init__

Initializes instance attributes after object creation.

self

Refers to the current instance inside instance methods.

Inheritance

A child class can inherit behavior and attributes from a parent class.

Encapsulation

Groups data and behavior and can control access through conventions and properties.

Polymorphism

The same interface or operation can work with objects of different types.

Abstraction

Focuses on essential behavior while hiding implementation details.

Overriding

A child class can provide a different implementation of an inherited method.

14. Exceptions

Exception

An exception interrupts normal execution unless it is handled.

try/except

try contains risky code; except handles selected exceptions.

else

Runs when no exception occurs in try.

finally

Runs whether or not an exception occurs; useful for cleanup.

raise

Explicitly raises an exception.

Common exceptions

ValueError, TypeError, IndexError, KeyError, NameError, AttributeError, ZeroDivisionError, FileNotFoundError, and ImportError.

15. File Handling

open()

Use open(path, mode, encoding='utf-8') to open a file.

Modes

r read, w write/replace, a append, x create, b binary.

with

with open(...) automatically closes the file after the block.

Read

read(), readline(), and readlines().

Write

write() and writelines().

16. Modules, Packages and pip

Module

A .py file containing reusable Python code.

Import

Use import module or from module import name.

Package

A structure used to organize related Python modules.

__name__

A directly executed script normally has __name__ equal to '__main__'.

Main guard

if __name__ == '__main__': keeps script-only code from running during import.

pip

Python's package installer.

requirements.txt

Lists project dependencies and versions; install using python -m pip install -r requirements.txt.

Virtual environment

An isolated environment for project dependencies.

17. Iterators and Generators

Iterable

An object that can be iterated over, commonly with for.

Iterator

An object providing __iter__() and __next__().

Generator

A function using yield that produces values lazily.

Benefit of generators

They can reduce memory use by producing values one at a time.

18. Decorators

Decorator

A callable that modifies or extends another function/class without changing its core source.

19. Regular Expressions

Regex

A pattern language for finding and manipulating text.

re functions

search, match, fullmatch, findall, finditer, sub, and split.

Raw strings

Regex patterns are often written as raw strings such as `r'\d+'`.

20. JSON

JSON

A text format widely used for structured data exchange.

json.dumps

Converts a Python object to a JSON string.

json.loads

Converts a JSON string to a Python object.

json.dump/json.load

Write/read JSON using file objects.

21. APIs and HTTP

API

An interface that allows software components to communicate.

HTTP methods

GET retrieves, POST submits/creates, PUT commonly replaces, PATCH partially updates, DELETE removes.

Status codes

200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error.

requests

The requests library is commonly used for HTTP calls in Python.

22. SQL and Databases

Database

A system for storing and retrieving organized data.

SQL

A language used to work with relational databases.

CRUD

Create, Read, Update, Delete.

Primary key

Uniquely identifies rows.

Foreign key

References a key in another table.

JOIN

Combines related rows from multiple tables.

SQL injection

A security problem caused by unsafe construction of SQL commands. Parameterized queries help prevent it.

23. Flask

Flask

A lightweight Python web framework.

Route

Maps a URL and HTTP method to a Python function.

request

Provides access to request data such as query parameters, form data, JSON, and headers.

JSON API

A Flask endpoint can return JSON data for client applications.

24. Testing and Debugging

Unit test

Tests a small unit of functionality in isolation.

pytest

A popular Python testing framework.

assert

Checks a condition and raises AssertionError when false.

Traceback

Shows where an exception occurred and the call path leading to it.

Debugger

Allows execution to be paused and inspected, often using breakpoints.

25. Common Python Errors

SyntaxError

The code cannot be parsed because its syntax is invalid.

IndentationError

Indentation is invalid or inconsistent.

NameError

A name is not defined.

TypeError

An operation received an incompatible type.

ValueError

A value has the correct general type but an inappropriate value.

IndexError

A sequence index is outside its valid range.

KeyError

A dictionary key is missing.

AttributeError

An object does not have the requested attribute.

ZeroDivisionError

Division or modulo by zero was attempted.

26. Important Interview Comparisons

== vs is

== compares values; is checks object identity.

sort vs sorted

list.sort() changes the list and returns None; sorted() returns a new sorted list.

append vs extend

append adds one item; extend adds elements from an iterable.

remove vs pop

remove deletes by value; pop deletes by index and returns the removed value.

pass vs continue

pass does nothing; continue skips to the next loop iteration.

shallow vs deep copy

A shallow copy may share nested objects; copy.deepcopy recursively copies nested objects.

27. Python Coding Patterns

Reverse a string

```
text[::-1]
```

Palindrome

```
text == text[::-1]
```

Even number

```
n % 2 == 0
```

List of evens

```
[x for x in numbers if x % 2 == 0]
```

Maximum

```
max(numbers)
```

Sum

```
sum(numbers)
```

Frequency

```
from collections import Counter; Counter(values)
```

Remove duplicates preserving order

```
list(dict.fromkeys(values))
```

Multiple integers

```
numbers = list(map(int, input().split()))
```

Enumerate

```
for index, value in enumerate(values): ...
```

Zip

```
for a, b in zip(list_a, list_b): ...
```

28. RAG and Chatbot Concepts

RAG

Retrieval-Augmented Generation retrieves relevant external information and supplies it to an LLM as context.

Document ingestion

Load documents, extract text, split into chunks, create embeddings, and store vectors.

Chunking

Divides long documents into smaller pieces for focused retrieval.

Embedding

A numerical vector representation of text or another object.

Vector database

Stores and indexes vectors for similarity search.

Retriever

Finds relevant chunks for a user query.

Prompt

Instructions and context sent to a language model.

Grounded answer

An answer based on retrieved context rather than unsupported generation.

Hallucination

Generated information that is unsupported or incorrect.

Pipeline

PDF → text → chunks → embeddings → vector store → retriever → context → LLM → answer.

Example: 5. Strings

```
s = "Python"
print(s[0])      # P
print(s[-1])     # n
print(s[1:4])    # yth
print("-".join(s.split()))
```

Example: 6. Lists

```
numbers = [3, 1, 2]
numbers.append(4)
numbers.sort()
print(numbers)

squares = [x * x for x in range(5)]
```

```
print(squares)
```

Example: 11. Functions

```
def add(a, b):  
    return a + b  
  
print(add(2, 3))  
  
def total(*args):  
    return sum(args)  
  
def show(**kwargs):  
    return kwargs
```

Example: 13. Object-Oriented Programming

```
class Student:  
    def __init__(self, name, marks):  
        self.name = name  
        self.marks = marks  
  
    def display(self):  
        print(self.name, self.marks)  
  
student = Student("Meghana", 85)  
student.display()
```

Example: 14. Exceptions

```
try:  
    number = int(input())  
    print(10 / number)  
except ValueError:  
    print("Enter a valid integer.")  
except ZeroDivisionError:  
    print("Cannot divide by zero.")  
finally:  
    print("Finished.")
```

Example: 15. File Handling

```
with open("example.txt", "w", encoding="utf-8") as file:  
    file.write("Hello Python")  
  
with open("example.txt", "r", encoding="utf-8") as file:  
    print(file.read())
```

Example: 18. Decorators

```
def logger(func):  
    def wrapper(*args, **kwargs):  
        print("Function started")  
        result = func(*args, **kwargs)  
        print("Function finished")  
        return result  
    return wrapper  
  
@logger  
def greet(name):  
    print("Hello", name)  
  
greet("Meghana")
```

Example: 28. RAG and Chatbot Concepts

```
# Conceptual RAG pipeline:  
# 1. Load PDF/document  
# 2. Extract text  
# 3. Split into chunks  
# 4. Create embeddings  
# 5. Store embeddings  
# 6. Retrieve relevant chunks  
# 7. Build prompt with retrieved context  
# 8. Send prompt to LLM  
# 9. Return grounded answer
```

29. Frequently Asked Python Questions

How do I reverse a string?

Use `text[::-1]`.

How do I check a palindrome?

Use `text == text[::-1]`.

How do I remove duplicates from a list?

Use `set(values)` if order does not matter. Use `dict.fromkeys(values)` when preserving insertion order is required.

How do I count characters?

Use `collections.Counter(text)`.

How do I find the largest number?

Use `max(numbers)`.

How do I read multiple integers?

Use `list(map(int, input().split()))`.

What does `enumerate()` do?

It returns index-value pairs while iterating.

What does `zip()` do?

It combines iterables position by position.

What does `map()` do?

It applies a function to each item and returns an iterator.

What does `filter()` do?

It keeps items for which a function returns true.

What is recursion?

A function calling itself with a base case that stops recursion.

What is Big O?

A way to describe how algorithm resource usage grows with input size.

What is PEP 8?

Python's main style guide.

What is a virtual environment?

An isolated environment for project dependencies.

What is Flask?

A lightweight Python web framework.

What is RAG?

Retrieval-Augmented Generation retrieves relevant context before generating an answer.

Why chunk documents?

To make retrieval focused and keep relevant context manageable.

30. Mini Programs

Even or Odd

```
n = int(input())
print("Even" if n % 2 == 0 else "Odd")
```

Factorial

```
n = int(input())
result = 1
for i in range(1, n + 1):
    result *= i
print(result)
```

Prime

```
n = int(input())
if n < 2:
    print("Not Prime")
else:
    prime = True
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            prime = False
            break
    print("Prime" if prime else "Not Prime")
```

Fibonacci

```
n = int(input())
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Vowel Count

```
text = input()
vowels = "aeiouAEIOU"
print(sum(1 for ch in text if ch in vowels))
```

Character Frequency

```
from collections import Counter
text = input()
print(Counter(text))
```

Sum of Digits

```
n = input()
print(sum(int(digit) for digit in n))
```

31. RAG Knowledge-Base Design Notes

- Use meaningful headings because they help both semantic retrieval and keyword matching.
- Keep each chunk focused on one concept, question, error, or code pattern.
- Store metadata such as topic, section, difficulty, and source when creating vector records.
- For coding questions, retrieve explanations and examples together when possible.
- For error questions, retrieve the error meaning, likely cause, and corrected pattern.
- Use source citations in your chatbot if you want traceable answers.
- For version-specific behavior, verify against the official Python documentation for the version being used.